There typically is no common usage across domains regarding score functions. The choice of score function for a particular task is most often based on empirical experiments. However, there are exceptions when, for example, efficiency is vital. In models where this is the case, the multiplicative or scaled multiplicative score functions are typically the best choice. An example of this is the Transformer model, which is generally computationally expensive.

3.2.2 Attention Alignment

The attention alignment is the step after the attention scoring. This alignment process directly determines which parts of the input data the model will attend to. The alignment function is denoted as align() and has various forms. The align() function takes as input the previously calculated attention score vector $\boldsymbol{e}$ and calculates for each element e_l of $\boldsymbol{e}$ the attention weight a_l. These attention weights can then be used to create the context vector $\boldsymbol{c}$ by taking a weighted average of the value vectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_{n_f}$:

$$\underset{d_v \times 1}{\boldsymbol{c}} = \sum_{l=1}^{n_f} \underset{1\times 1}{a_l} \times \underset{d_v \times 1}{\boldsymbol{v}_l}. \tag{24}$$

The most popular alignment method to calculate these weights is a simple softmax function, as depicted in (25).

$$\underset{1\times 1}{a_l} = \text{align}(\underset{1\times 1}{e_l}; \underset{n_f \times 1}{\boldsymbol{e}}) = \frac{\exp(e_l)}{\sum_{j=1}^{n_f} \exp(e_j)}. \tag{25}$$

This alignment method is often referred to as **soft alignment** in computer vision settings [8], or **global alignment** for sequence data [4]. Nevertheless, both these terms represent the same function and can be interpreted similarly. Soft/global alignment can be interpreted as the model attending to all feature vectors. For example, the model attends to all regions in an image, or all words in a sentence. Even though the attention model generally does focus more on specific parts of the input, every part of the input will receive at least some amount of attention due to the nature of the softmax function. Furthermore, an advantage of the softmax function is that it introduces a probabilistic interpretation to the input vectors. This allows one to easily analyze which parts of the input are important to the output predictions.

In contrast to soft/global alignment, other methods aim to achieve a more focused form of alignment. For example, **hard alignment** [8], also known as hard attention or non-deterministic attention, is an alignment type that forces the attention model to focus on exactly one feature vector. Firstly, this method implements the softmax function in the exact same way as global alignment. However, the outputs $a_1, \ldots, a_{n_f}$ are not used as weights for the context vector calculation. Instead, these values are used as probabilities to draw the choice of the one value vector from. A value $m \in \mathbb{R}^1$ is drawn from a multinomial distribution with $a_1, \ldots, a_{n_f}$ as parameters for the probabilities. Then, the context vector is simply defined as follows:

$$\underset{d_v \times 1}{\boldsymbol{c}} = \underset{d_v \times 1}{\boldsymbol{v}_m}. \tag{26}$$

Hard alignment is typically more efficient at inference compared to soft alignment. On the other hand, the main disadvantage of hard attention is that, due to the stochastic alignment of attention, the training of the model cannot be done via the regular backpropagation method. Instead, simulation and sampling, or reinforcement learning [57] are required to calculate the gradient at the hard attention layer. As such, soft/global attention is generally preferred. However, a compromise can be made in certain situations. **Local alignment** [4] is a method that implements a softmax distribution, similarly to soft/global alignment. But, the softmax distribution is calculated based only on a subset of the inputs. This method is generally used in combination with sequence data. One has to specify a variable $p \in \mathbb{R}^1$ that determines the position of the region. Feature vectors close to p will be attended to by the model, and vectors too far from p will be ignored. The size of the subset will be determined by the variable $D \in \mathbb{R}^1$. Summarizing, the attention model will apply a softmax function on the attention scores in the subset $[p-D, p+D]$. In other words, a window is placed on the input and soft/global attention is calculated within that window:

$$\underset{1\times 1}{a_l} = \text{align}(\underset{1\times 1}{e_l}; \underset{n_f \times 1}{\boldsymbol{e}}) = \frac{\exp(e_l)}{\sum_{j=p-D}^{p+D} \exp(e_j)}. \tag{27}$$

The question that remains is how to determine the location parameter p. The first method is referred to as **monotonic alignment**. This straightforward method entails simply setting the location parameter equal to the location of the prediction in the output sequence. Another method of determining the position of the region is referred to as **predictive alignment**. As the name entails, the model attempts to actually predict the location of interest in the sequence:

$$\underset{1\times 1}{p} = \underset{1\times 1}{S} \times \text{sigmoid}(\underset{1\times d_p}{\boldsymbol{w}_p^T} \times \tanh(\underset{d_p \times d_q}{\boldsymbol{W}_p} \times \underset{d_q \times 1}{\boldsymbol{q}})), \tag{28}$$

where $S \in \mathbb{R}^1$ is the length of the input sequence, and $\boldsymbol{w}_p \in \mathbb{R}^{d_p}$ and $\boldsymbol{W}_p \in \mathbb{R}^{d_p \times d_q}$ are both trainable weights parameters. The sigmoid function multiplied by S makes sure that p is in the range $[0, S]$. Additionally, in [4], it is recommended to add an additional term to the alignment function to favor alignment around p:

$$\underset{1\times 1}{a_l} = \text{align}(\underset{1\times 1}{e_l}; \underset{n_f \times 1}{\boldsymbol{e}}) \exp(-\frac{(l-p)^2}{2\sigma^2}), \tag{29}$$

where $\sigma \in \mathbb{R}^1$ is empirically set equal to $\frac{D}{2}$ according to [4]. Another proposed method for compromising between soft and hard alignment is **reinforced alignment** [58]. Similarly to local alignment, a subset of the feature vectors is determined, for which soft alignment is calculated. However, instead of using a window to determine the subset, reinforced alignment uses a reinforcement learning agent [57], similarly to hard alignment, to choose the subset of feature vectors. The attention calculation based on these chosen feature vectors is the same as regular soft alignment.

Soft alignment is often regarded as the standard alignment function for attention models in practically every domain. Yet, the other alignment methods have also seen interesting uses in various domains. For example, hard attention is used in [59] for the task of visual question answering. In [60], both soft and hard attention are used in a graph attention model for multi-agent game abstraction. Similarly, in [61], both global and local alignment are used for review rating predictions. Reinforced alignment has been employed

in combination with a co-attention structure in [62] for the task of aspect sentiment classification. In [63], reinforced alignment is used for the task of person re-identification using surveillance images.

3.2.3 Attention Dimensionality

All previous model specifications of attention use a scalar weight a_l for each value vector $\boldsymbol{v}_l$. This technique is referred to as **single-dimensional attention**. However, instead of determining a single attention score and weight for the entire vector, [64] proposes to calculate weights for every single feature in those vectors separately. This technique is referred to as **multi-dimensional attention**, since the attention weights now become higher dimensional vectors. The idea is that the model no longer has to attend to entire vectors, but it can instead pick and choose specific elements from those vectors. More specifically, attention is calculated for each dimension. As such, the model must create a vector of attention weights $\boldsymbol{a}_l \in \mathbb{R}^{d_v}$ for each value vector $\boldsymbol{v}_l \in \mathbb{R}^{d_v}$. The context vector can then be calculated by summing the element-wise multiplications ($\circ$) of the value vectors $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_{n_f} \in \mathbb{R}^{d_v}$ and the corresponding attention weight vectors $\boldsymbol{a}_1, \ldots, \boldsymbol{a}_{n_f} \in \mathbb{R}^{d_v}$, as follows:

$$\underset{d_v \times 1}{\boldsymbol{c}} = \sum_{l=1}^{n_f} \underset{d_v \times 1}{\boldsymbol{a}_l} \circ \underset{d_v \times 1}{\boldsymbol{v}_l} . \tag{30}$$

However, since one needs to create attention weight vectors, this technique requires adjusted attention score and weight calculations. For example, the concatenate score function found in Table 2 can be adjusted by changing the $\boldsymbol{w} \in \mathbb{R}^{d_w}$ weights vector to the weight matrix $\boldsymbol{W}_d \in \mathbb{R}^{d_w \times d_v}$:

$$\underset{d_v \times 1}{\boldsymbol{e}_l} = \underset{d_v \times d_w}{\boldsymbol{W}_d^T} \times \text{act}(\underset{d_w \times d_q}{\boldsymbol{W}_1} \times \underset{d_q \times 1}{\boldsymbol{q}} + \underset{d_w \times d_k}{\boldsymbol{W}_2} \times \underset{d_k \times 1}{\boldsymbol{k}_l} + \underset{d_w \times 1}{\boldsymbol{b}}). \tag{31}$$

This new score function produces the attention score vectors $\boldsymbol{e}_1, \ldots, \boldsymbol{e}_{n_f} \in \mathbb{R}^{d_v}$. These score vectors can be combined into a matrix of scores $\boldsymbol{e} = [\boldsymbol{e}_1, \ldots, \boldsymbol{e}_{n_f}] \in \mathbb{R}^{d_v \times n_f}$. To produce multi-dimensional attention weights, the alignment function stays the same, but it is applied for each feature across the attention score columns. To illustrate, when implementing soft attention, the attention weight produced from the ith element of score vector $\boldsymbol{e}_l$ is defined as follows:

$$\underset{1 \times 1}{a_{l,i}} = \text{align}(\underset{1 \times 1}{e_{l,i}}; \underset{d_v \times n_f}{\boldsymbol{e}}) = \frac{\exp(e_{l,i})}{\sum_{j=1}^{n_f} \exp(e_{j,i})}, \tag{32}$$

where $e_{l,i}$ represents the ith element of score vector $\boldsymbol{e}_l$, and $a_{l,i}$ is the ith element of the attention weights vector $\boldsymbol{a}_l$. Finally, these attention weight vectors can be used to compute the context vector as presented in (30).

Multi-dimensional attention is a very general mechanism that can be applied in practically every attention model, but actual applications of the technique have been relatively sparse. One application example is [65], where multi-dimensional attention is used in a model for named entity recognition based on text and visual context from multimedia posts. In [66], multi-dimensional attention is used in a model for answer selection in community question answering. In [67], the U-net model for medical image segmentation is extended with a multi-dimensional attention mechanism. Similarly, in [68], the Transformer model is extended with the multi-dimensional attention mechanism for the task of dialogue response generation. In [69], multi-dimensional attention is used to extend graph attention networks for dialogue state tracking. Lastly, for the task of next-item recommendation, [70] proposes a model that incorporates multi-dimensional attention.

3.3 Query-Related Attention Mechanisms

Queries are an important part of any attention model, since they directly determine which information is extracted from the feature vectors. These queries are based on the desired output of the task model, and can be interpreted as literal questions. Some queries have specific characteristics that require specific types of mechanisms to process them. As such, this category encapsulates the attention mechanisms that deal with specific types of query characteristics. The mechanisms in this category deal with one of the two following query characteristics: the type of queries or the multiplicity of queries.

3.3.1 Type of Queries

Different attention models employ attention for different purposes, meaning that distinct query types are necessary. There are **basic queries**, which are queries that are typically straightforward to define based on the data and model. For example, the hidden state for one prediction in an RNN is often used as the query for the next prediction. One could also use a vector of auxiliary variables as query. For example, when doing medical image classification, general patient characteristics can be incorporated into a query.

Some attention mechanisms, such as co-attention, rotatory attention, and attention-over-attention, use **specialized queries**. For example, rotatory attention uses the context vector from another attention module as query, while interactive co-attention uses an averaged keys vector based on another input. Another case one can consider is when attention is calculated based purely on the feature vectors. This concept has been mentioned before and is referred to as **self-attention** or **intra-attention** [71]. We say that the models use **self-attentive queries**. There are two ways of interpreting such queries. Firstly, one can say that the query is constant. For example, document classification requires only a single classification as the output of the model. As such, the query is always the same, namely: "What is the class of the document?". The query can be ignored and attention can be calculated based only on the features themselves. Score functions can be adjusted for this by making the query vector a vector of constants or removing it entirely:

$$\text{score}(\underset{d_k \times 1}{\boldsymbol{k}_l}) = \underset{1 \times d_w}{\boldsymbol{w}^T} \times \text{act}(\underset{d_w \times d_k}{\boldsymbol{W}} \times \underset{d_k \times 1}{\boldsymbol{k}_l} + \underset{d_w \times 1}{\boldsymbol{b}}). \tag{33}$$

Additionally, one can also interpret self-attention as learning the query along the way, meaning that the query can be defined as a trainable vector of weights. For example, the dot-product score function may take the following form:

$$\text{score}(\underset{d_k \times 1}{\boldsymbol{k}_l}) = \underset{1 \times d_k}{\boldsymbol{q}^T} \times \underset{d_k \times 1}{\boldsymbol{k}_l} , \tag{34}$$

where $\boldsymbol{q} \in \mathbb{R}^{d_k}$ is a trainable vector of weights. One could also interpret vector $\boldsymbol{b} \in \mathbb{R}^{d_w}$ as the query in (33). Another

use of self-attention is to uncover the relations between the feature vectors $\boldsymbol{f}_1, \ldots, \boldsymbol{f}_{n_f}$. These relations can then be used as additional information to incorporate into new representations of the feature vectors. With basic attention mechanisms, the keys matrix $\boldsymbol{K}$, and the values matrix $\boldsymbol{V}$ are extracted from the features matrix $\boldsymbol{F}$, while the query $\boldsymbol{q}$ is produced separately. For this type of self-attention, the query vectors are extracted in a similar process as the keys and values, via a transformation matrix of trainable weights $\boldsymbol{W}_Q \in \mathbb{R}^{d_q \times d_f}$. We define the matrix $\boldsymbol{Q} = [\boldsymbol{q}_1, \ldots, \boldsymbol{q}_{n_f}] \in \mathbb{R}^{d_q \times n_f}$, which can be obtained as follows:

$$\underset{d_q \times n_f}{\boldsymbol{Q}} = \underset{d_q \times d_f}{\boldsymbol{W}_Q} \times \underset{d_f \times n_f}{\boldsymbol{F}} . \tag{35}$$

Each column of $\boldsymbol{Q}$ can be used as the query for the attention model. When attention is calculated using a query $\boldsymbol{q}$, the resulting context vector $\boldsymbol{c}$ will summarize the information in the feature vectors that is important to the query. Since the query, or a column of $\boldsymbol{Q}$, is now also a feature vector representation, the context vector contains the information of all feature vectors that are important to that specific feature vector. In other words, the context vectors capture the relations between the feature vectors. For example, self-attention allows one to extract the relations between words: which verbs refer to which nouns, which pronouns refer to which nouns, etc. For images, self-attention can be used to determine which image regions relate to each other.

While self-attention is placed in the query-related category, it is also very much related to the feature model. Namely, self-attention is a technique that is often used in the feature model to create improved representations of the feature vectors. For example, the Transformer model for language processing [13], and the Transformer model for image processing [15], both use multiple rounds of (multi-head) self-attention to improve the representation of the feature vectors. The relations captured by the self-attention mechanism are incorporated into new representations. A simple method of determining such a new representation is to simply set the feature vectors equal to the acquired self-attention context vectors [71], as presented in (36).

$$\underset{d_f \times 1}{\boldsymbol{f}^{(\text{new})}} = \underset{d_f \times 1}{\boldsymbol{c}} , \tag{36}$$

where $\boldsymbol{f}^{(\text{new})}$ is the updated feature vector. Another possibility is to add the context vectors to the previous feature vectors with an additional normalization layer [13]:

$$\underset{d_f \times 1}{\boldsymbol{f}^{(\text{new})}} = \text{Normalize}(\underset{d_f \times 1}{\boldsymbol{f}^{(\text{old})}} + \underset{d_f \times 1}{\boldsymbol{c}}), \tag{37}$$

where $\boldsymbol{f}^{(\text{old})}$ is the previous feature vector, and Normalize() is a normalization layer [72]. Using such techniques, self-attention has been used to create improved word or sentence embeddings that enhance model accuracy [71].

Self-attention is arguably one of the more important types of attention, partly due to its vital role in the highly popular Transformer model. Self-attention is a very general mechanism and can be applied to practically any problem. As such, self-attention has been extensively explored in many different fields in both Transformer-based architectures and other types of models. For example, in [73], self-attention is explored for image recognition tasks, and results indicate that the technique may have substantial advantages with regards to robustness and generalization. In [74], self-attention is used in a generative adversarial network (GAN) [75] to determine which regions of the input image to focus on when generating the regions of a new image. In [76], self-attention is used to design a state-of-the-art medical image segmentation model. Naturally, self-attention can also be used for video processing. In [77], a self-attention model is proposed for the purpose of video summarization that reaches state-of-the-art results. In other fields, like audio processing, self-attention has been explored as well. In [78], self-attention is used to create a speech recognition model. Self-attention has also been explored in overlapping domains. For example, in [79], the self-attention Transformer architecture is used to create a model that can recognize phrases from audio and by lip-reading from a video. For the problem of next item recommendation, [80] proposes a Transformer model that explicitly captures item-item relations using self-attention. Self-attention also has applications in any natural language processing fields. For example, in [81], self-attention is used for sentiment analysis. Self-attention is also highly popular for graph models. For example, self-attention is explored in [82] for the purpose of representation learning in communication networks and rating networks. Additionally, the first attention model for graph networks was based on self-attention [83].

3.3.2 Multiplicity of Queries

In previous examples, the attention model generally used a single query for a prediction. We say that such models use **singular query attention**. However, there are attention architectures that allow the model to compute attention using multiple queries. Note that this is different from, for example, an RNN that may involve multiple queries to produce a sequence of predictions. Namely, such a model still requires only a single query per prediction.

One example of a technique that incorporates multiple queries is **multi-head attention** [13], as presented in Fig. 9. Multi-head attention works by implementing multiple attention modules in parallel by utilizing multiple different versions of the same query. The idea is to linearly transform the query $\boldsymbol{q}$ using different weight matrices. Each newly formed query essentially asks for a different type of relevant information, allowing the attention model to introduce more information into the context vector calculation. An attention model implements $d \geq 1$ heads with each attention head having its own query vector, keys matrix, and values matrix: $\boldsymbol{q}^{(j)}$, $\boldsymbol{K}^{(j)}$ and $\boldsymbol{V}^{(j)}$, for $j = 1, \ldots, d$. The query $\boldsymbol{q}^{(j)}$ is obtained by linearly transforming the original query $\boldsymbol{q}$, while the matrices $\boldsymbol{K}^{(j)}$ and $\boldsymbol{V}^{(j)}$ are obtained through linear transformations of $\boldsymbol{F}$. As such, each attention head has its own learnable weights matrices $\boldsymbol{W}_q^{(j)}$, $\boldsymbol{W}_K^{(j)}$ and $\boldsymbol{W}_V^{(j)}$ for these transformations. The calculation of the query, keys, and values for the jth head are defined as follows:

$$\underset{d_q \times 1}{\boldsymbol{q}^{(j)}} = \underset{d_q \times d_q}{\boldsymbol{W}_q^{(j)}} \times \underset{d_q \times 1}{\boldsymbol{q}} , \qquad \underset{d_k \times n_f}{\boldsymbol{K}^{(j)}} = \underset{d_k \times d_f}{\boldsymbol{W}_K^{(j)}} \times \underset{d_f \times n_f}{\boldsymbol{F}} , \qquad \underset{d_v \times n_f}{\boldsymbol{V}^{(j)}} = \underset{d_v \times d_f}{\boldsymbol{W}_V^{(j)}} \times \underset{d_f \times n_f}{\boldsymbol{F}} . \tag{38}$$

Thus, each head creates its own representations of the query $\boldsymbol{q}$, and the input matrix $\boldsymbol{F}$. Each head can therefore